

```

def _update_screen(self):
    """Update images on the screen, and flip to the new screen."""
    self.screen.fill(self.settings.bg_color)
    ❷ for bullet in self.bullets.sprites():
        bullet.draw_bullet()
    self.ship.blitme()

    pygame.display.flip()
--snip--

```

We call `_fire_bullet()` when the spacebar is pressed ❶. In `_fire_bullet()`, we make an instance of `Bullet` and call it `new_bullet` ❷. We then add it to the group `bullets` using the `add()` method ❸. The `add()` method is similar to `append()`, but it's written specifically for Pygame groups.

The `bullets.sprites()` method returns a list of all sprites in the group `bullets`. To draw all fired bullets to the screen, we loop through the sprites in `bullets` and call `draw_bullet()` on each one ❹. We place this loop before the line that draws the ship, so the bullets don't start out on top of the ship.

When you run *alien_invasion.py* now, you should be able to move the ship right and left and fire as many bullets as you want. The bullets travel up the screen and disappear when they reach the top, as shown in Figure 12-3. You can alter the size, color, and speed of the bullets in *settings.py*.

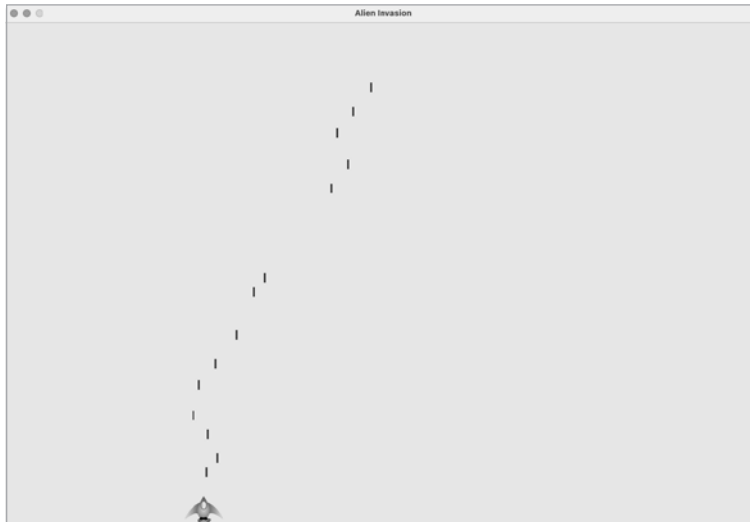


Figure 12-3: The ship after firing a series of bullets

Deleting Old Bullets

At the moment, the bullets disappear when they reach the top, but only because Pygame can't draw them above the top of the screen. The bullets actually continue to exist; their *y*-coordinate values just grow increasingly negative. This is a problem because they continue to consume memory and processing power.